Task S4.01. Database Creation

Mani Rezaeirad *p2p review partner: Damian Vallejos*

Level 1

Download the CSV files, study them and design a database with a star schema that contains at least 4 tables from which you can perform the following queries:

For starters, a database named **sprint4_bbdd** is created to host the tables that would be imported in the next step.

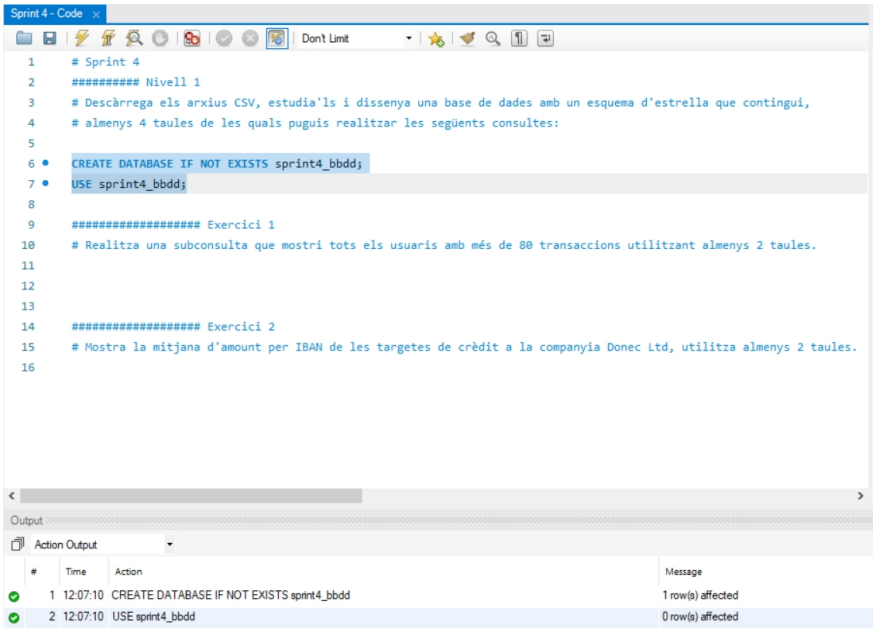

Fig 1. *Creating the database sprint4_bbdd*

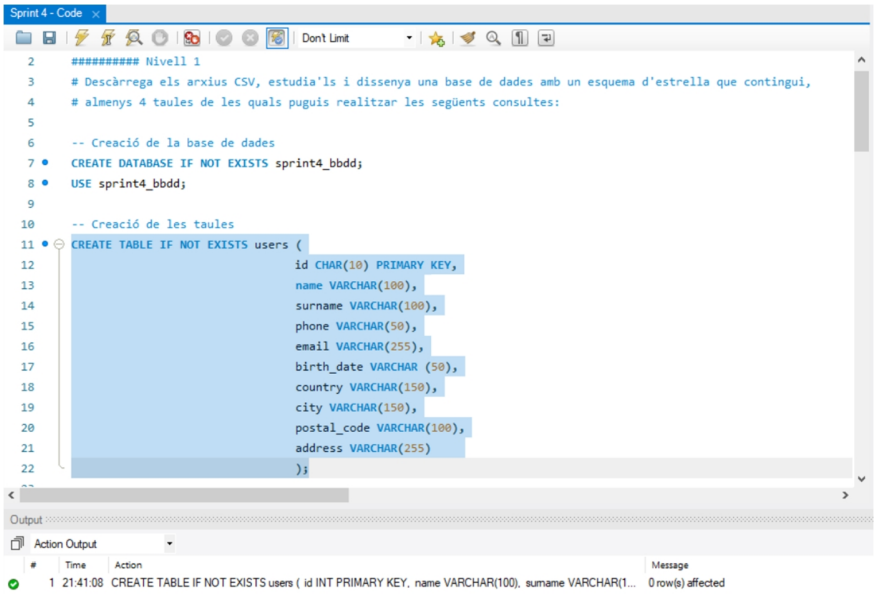

Fig 2. *Creating the users table*

Task S4.01. Database Creation

Mani Rezaeirad *p2p review partner: Damian Vallejos*

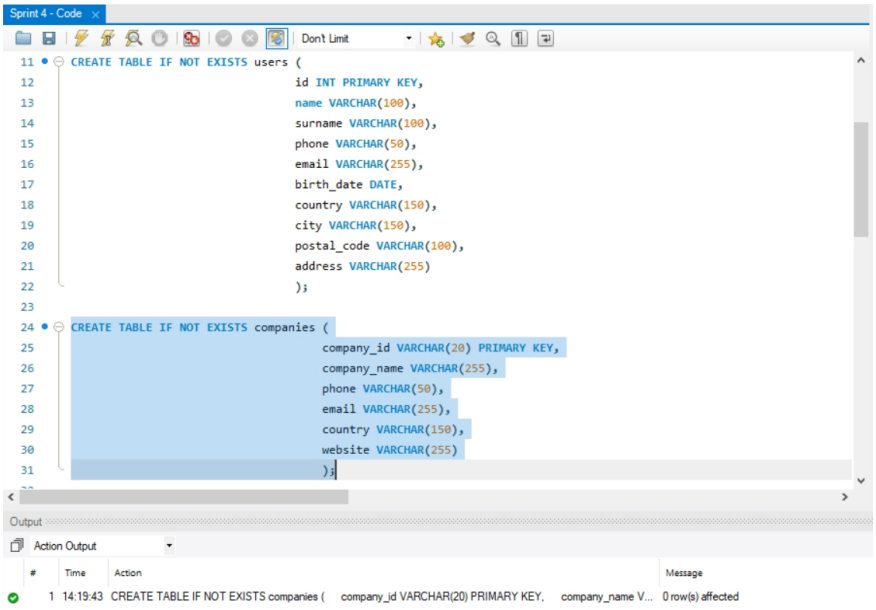

Fig 3. *Creating the* `companies` *table*

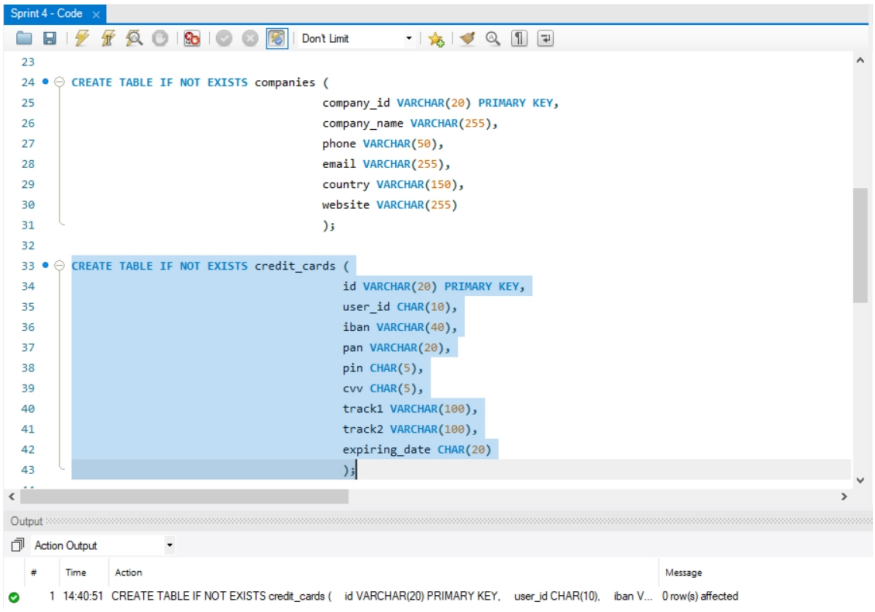

Fig 4. *Creating the* `credit_cards` *table*

Task S4.01. Database Creation

Mani Rezaeirad *p2p review partner: Damian Vallejos*

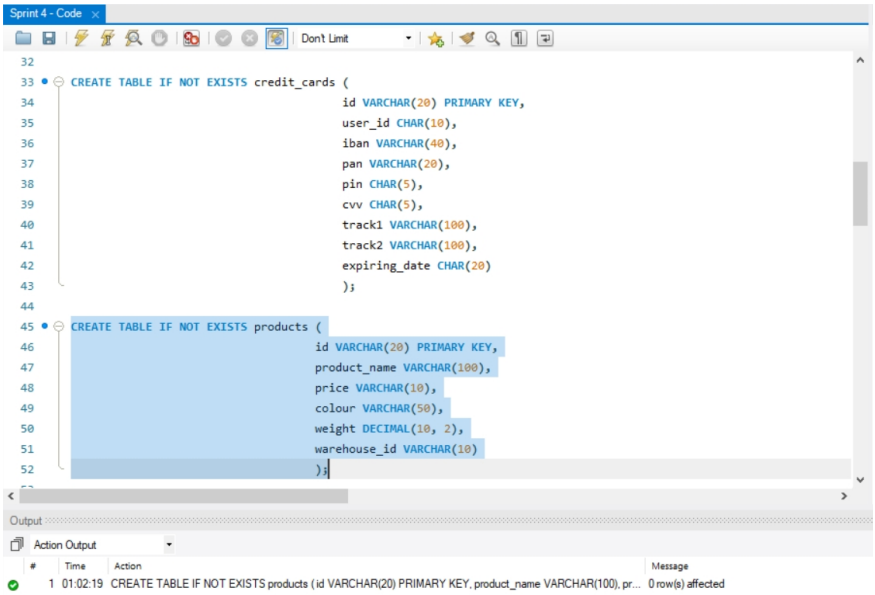

Fig 5. *Creating the* `products` *table*

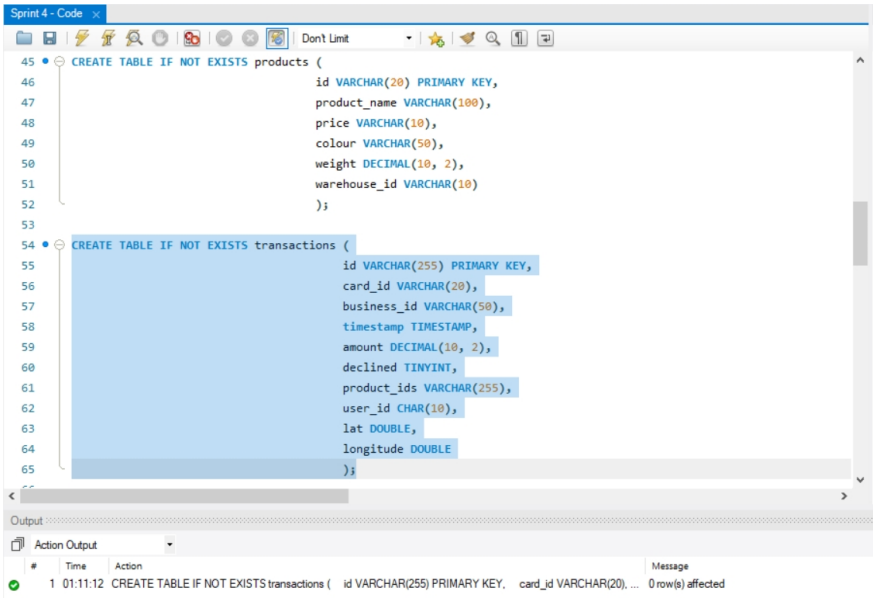

Fig 6. *Creating the* `transactions` *table*

Once the corresponding tables are created, it's time to import CSV files into them.

As shown in Figure 7, MySQL has a limitation that restricts the directories from which, files can be loaded. Figure 8 shows a command to display the configured directory. To solve this problem, there are two options: to move the CSV files to the specified directory or to disable the configuration. Since this change is just needed for this Sprint, it is better to simply move the CSV files to the designated directory. By doing so, the issue will be resolved, and loading the data can be done as shown in Figure 9 to Figure 14. Keep in mind that both CSV files, american_users and european_users, are imported into one single table: `users`.

Task S4.01. Database Creation

Mani Rezaeirad *p2p review partner: Damian Vallejos*

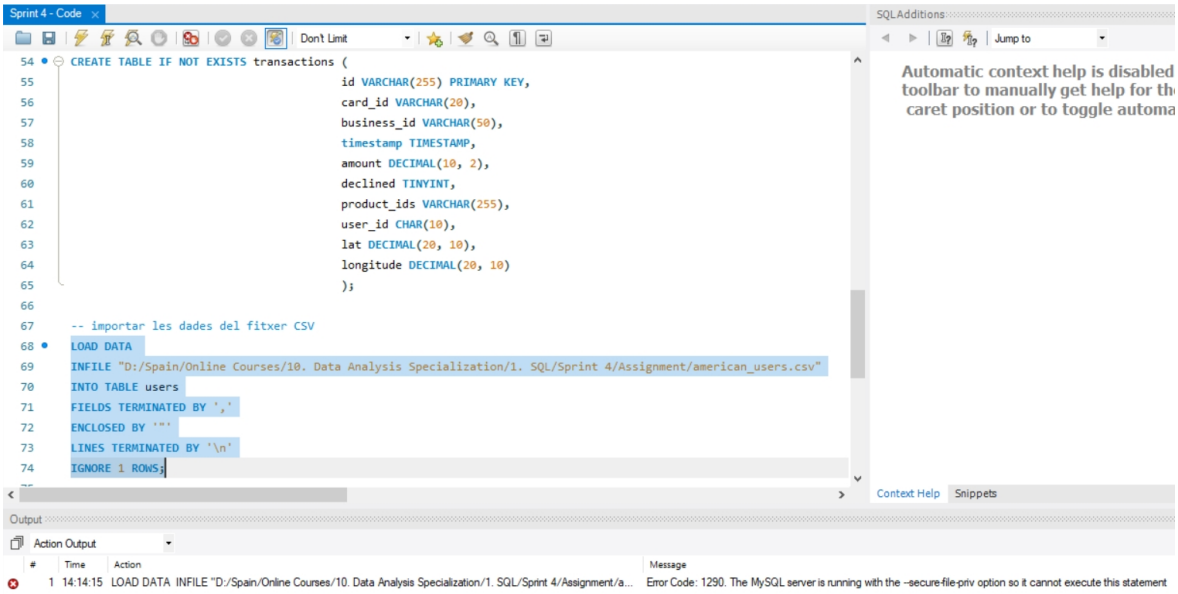

Fig 7. *The directory limit prevents the loading of data files*

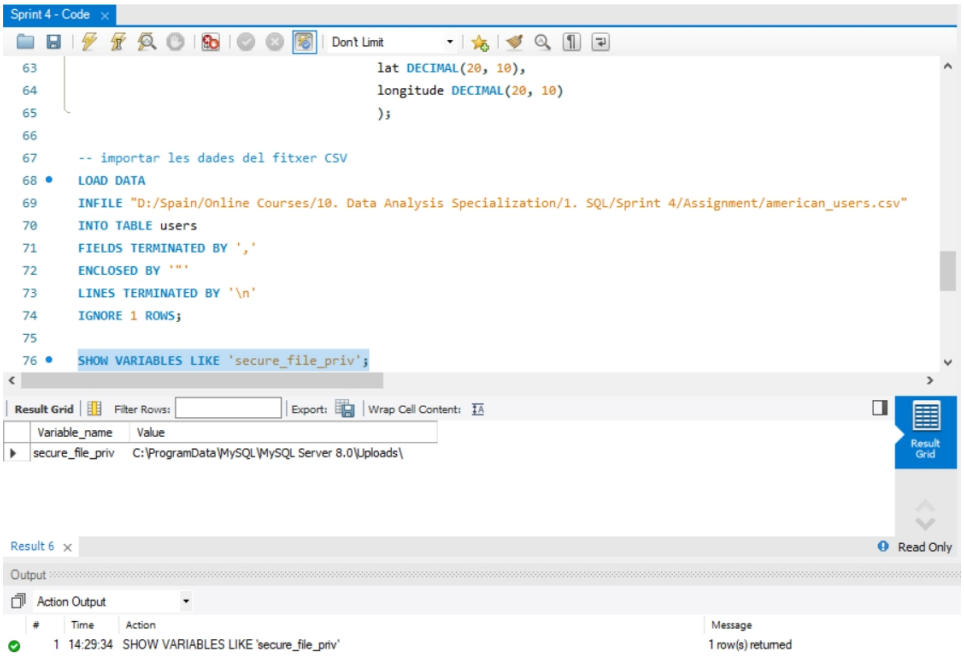

Fig 8. *Showing the configured directory*

Task S4.01. Database Creation

Mani Rezaeirad *p2p review partner: Damian Vallejos*

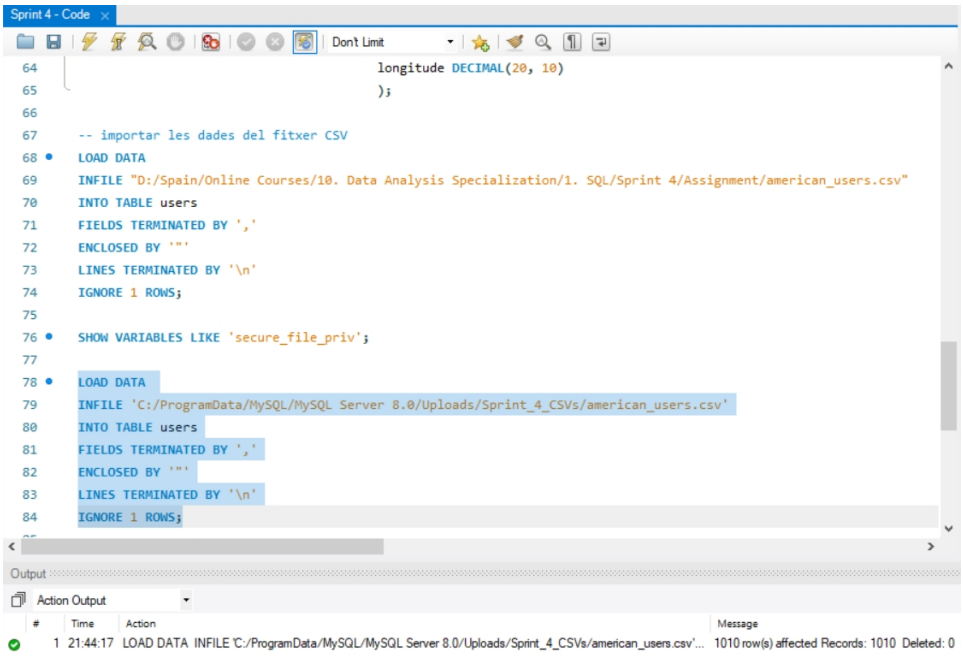

Fig 9. *Importing the CSV data of american_users into the users table*

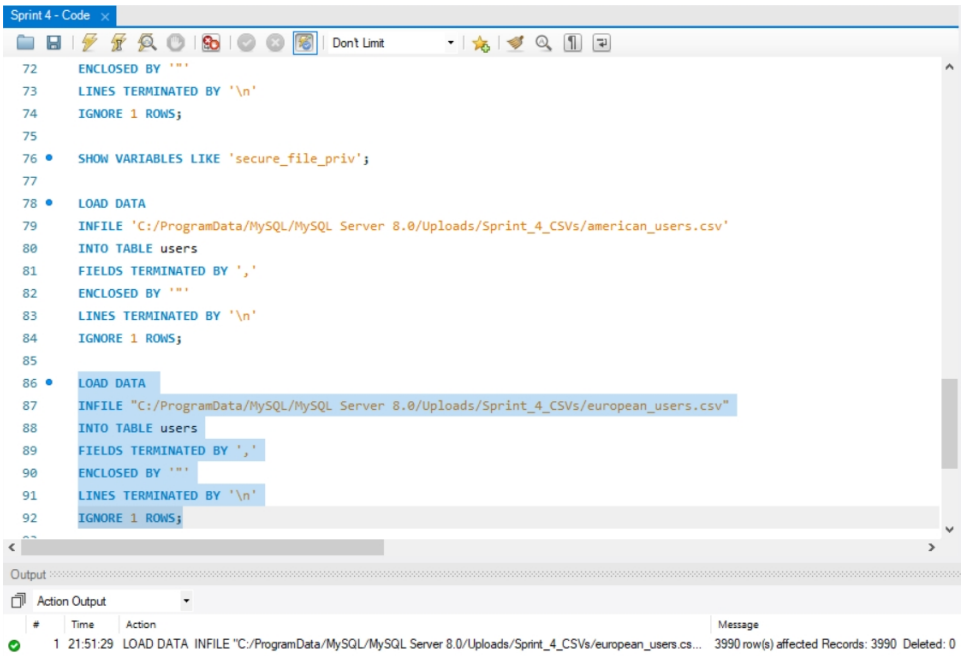

Fig 10. *Importing the CSV data of european_users into the users table*

Task S4.01. Database Creation

Mani Rezaeirad *p2p review partner: Damian Vallejos*

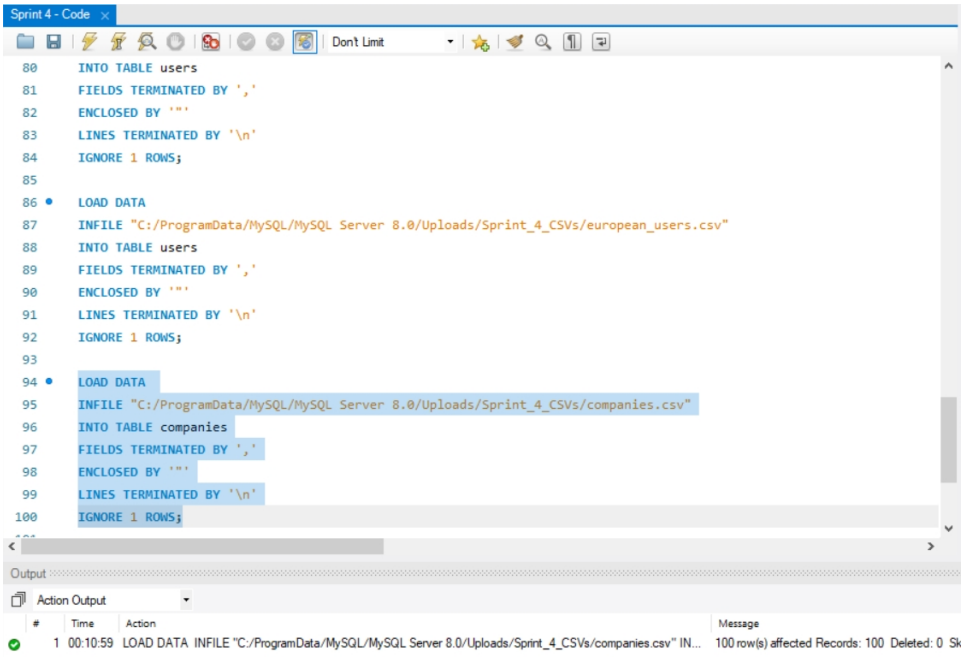

Fig 11. *Importing the CSV data of companies into the* `companies` *table*

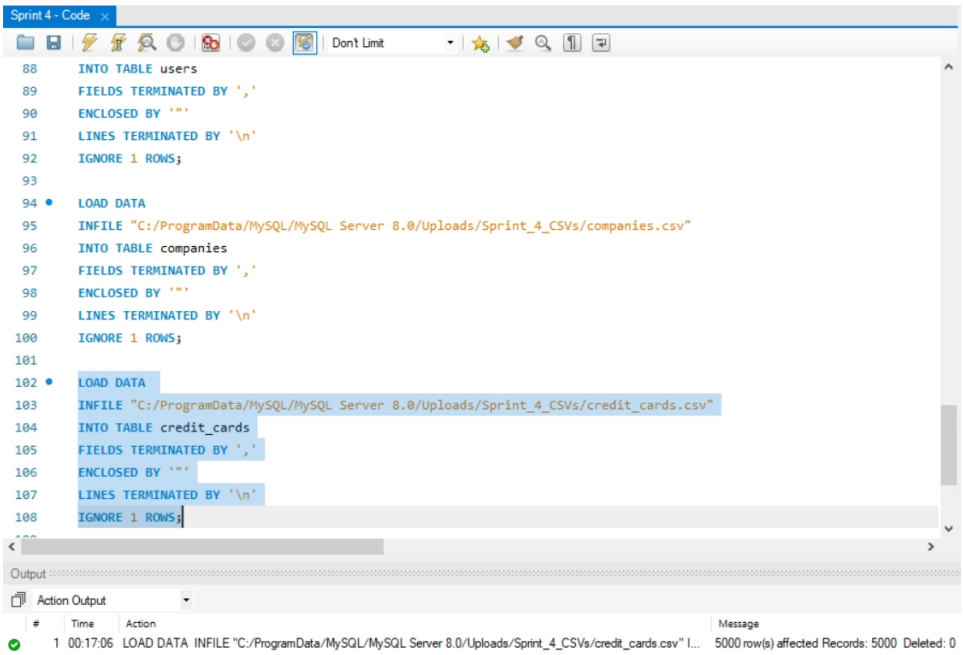

Fig 12. *Importing the CSV data of* `credit_cards` *into the* `credit_cards` *table*

Task S4.01. Database Creation

Mani Rezaeirad *p2p review partner: Damian Vallejos*

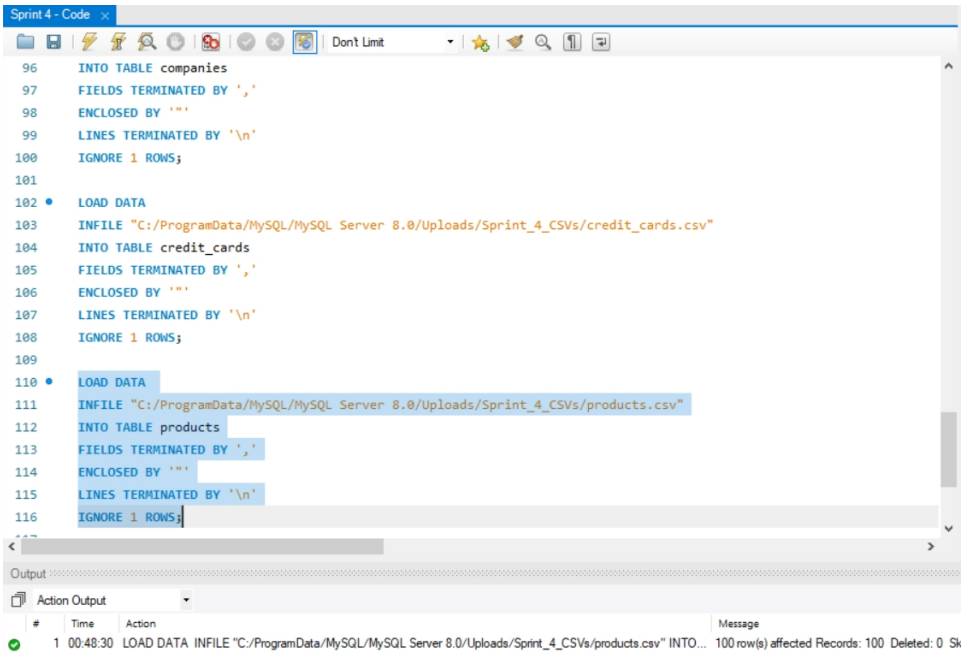

Fig 13. *Importing the CSV data of products into the* `products` *table*

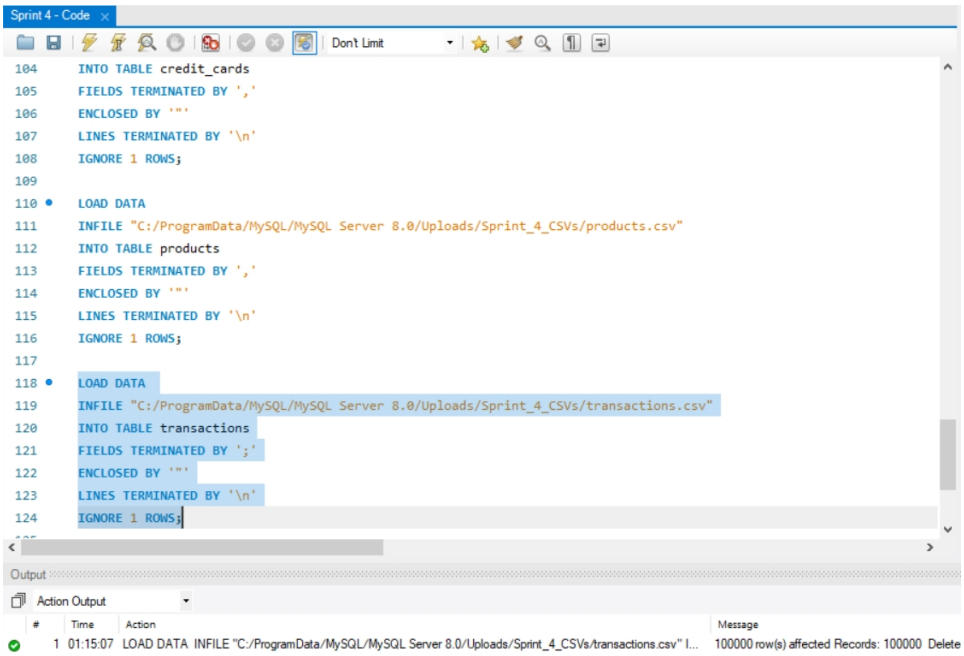

Fig 14. *Importing the CSV data of transactions into the* `transactions` *table*

To normalize the headers of each table for further use, some names have been changed, as shown in Figure 15.

Task S4.01. Database Creation

Mani Rezaeirad *p2p review partner: Damian Vallejos*

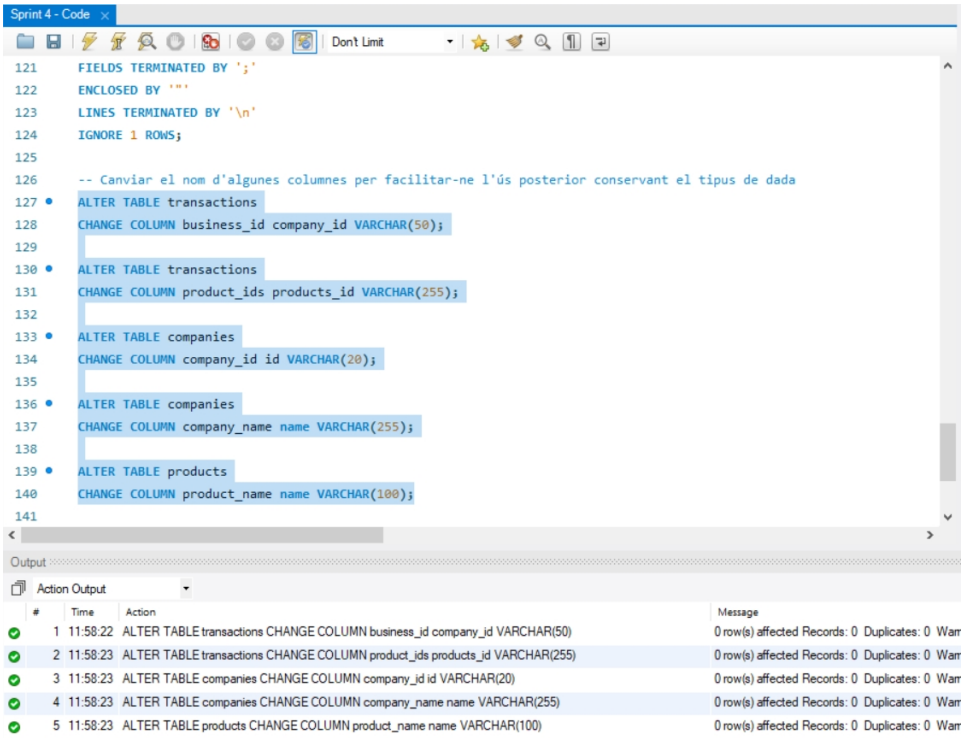

Fig 15. *Further use reference facilitation by changing some header names*

Now we link the `transaction` table to its corresponding tables, and also, the `credit_cards` table to the `users` table using PK-FK, as shown in Figure 16.

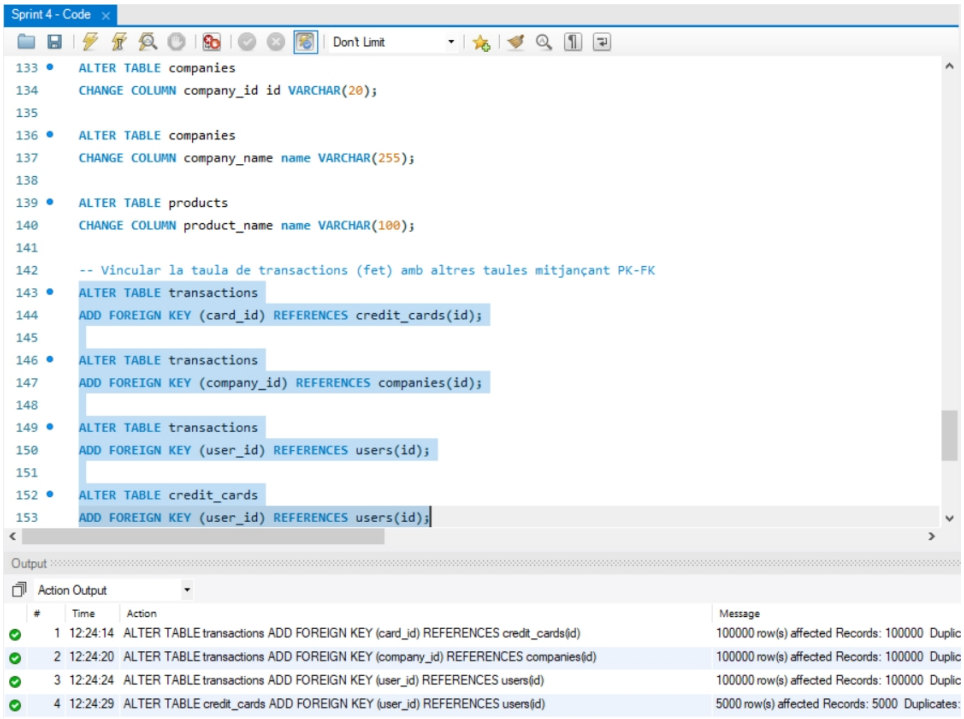

Fig 16. *Linking the `transactions` table to other tables using PK-FK*

Task S4.01. Database Creation

Mani Rezaeirad *p2p review partner: Damian Vallejos*

Before entering the first exercise, some last small database cleaning steps are done, as shown in Figure 17. To avoid the safe updates error, we turn updates off by setting it to 0, then set it back to 1 after running the code (NB! it is not recommended by many users, but it was the fastest way to solve the problem).

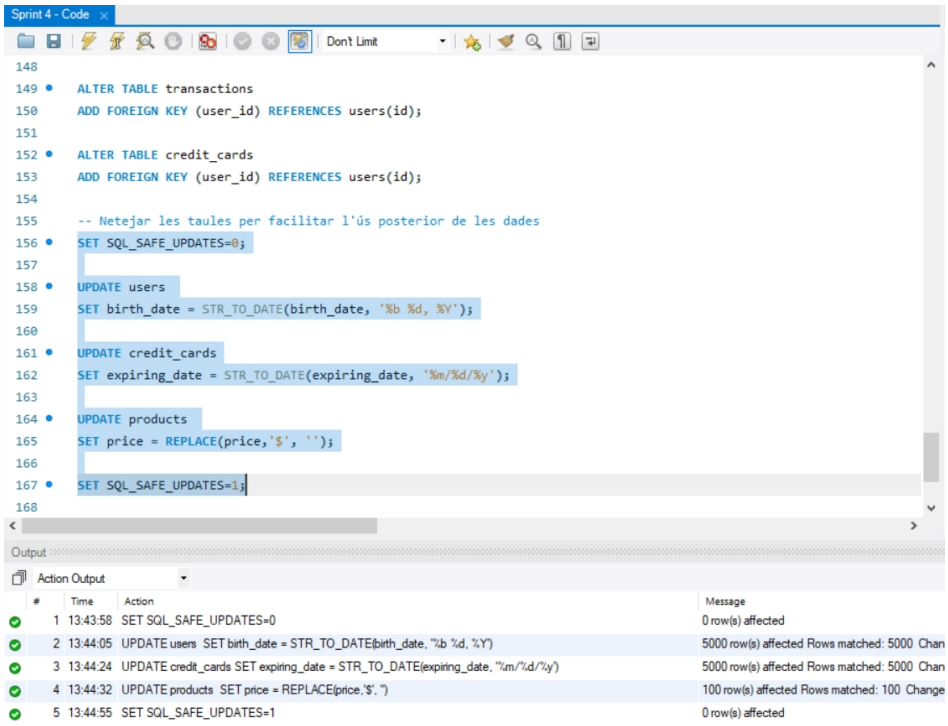

Fig 17. *Cleaning the database by changing some formats of some table columns*

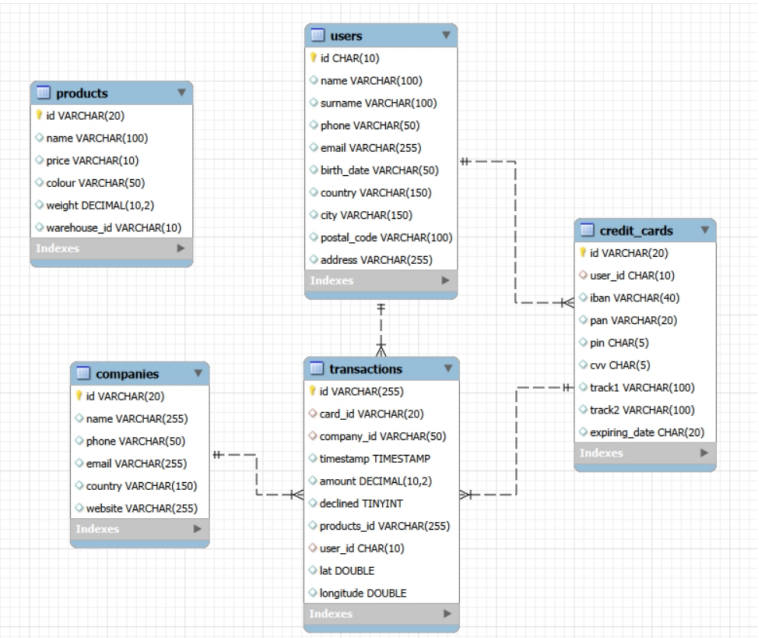

Fig 18. *ER diagram of the sprint4_bbdd database with a STAR schema*

Task S4.01. Database Creation

Mani Rezaeirad *p2p review partner: Damian Vallejos*

As can be seen in Figure 18, a STAR schema is depicted, with the `transactions` table (fact table) in the middle and the `users`, `credit_cards`, and `companies` tables (dimension tables) in a 1:N relationship with it. The `users` table also has a 1:N relationship with the `credit_cards` table.

Exercise 1

Perform a subquery that shows all users with more than 80 transactions using at least 2 tables.

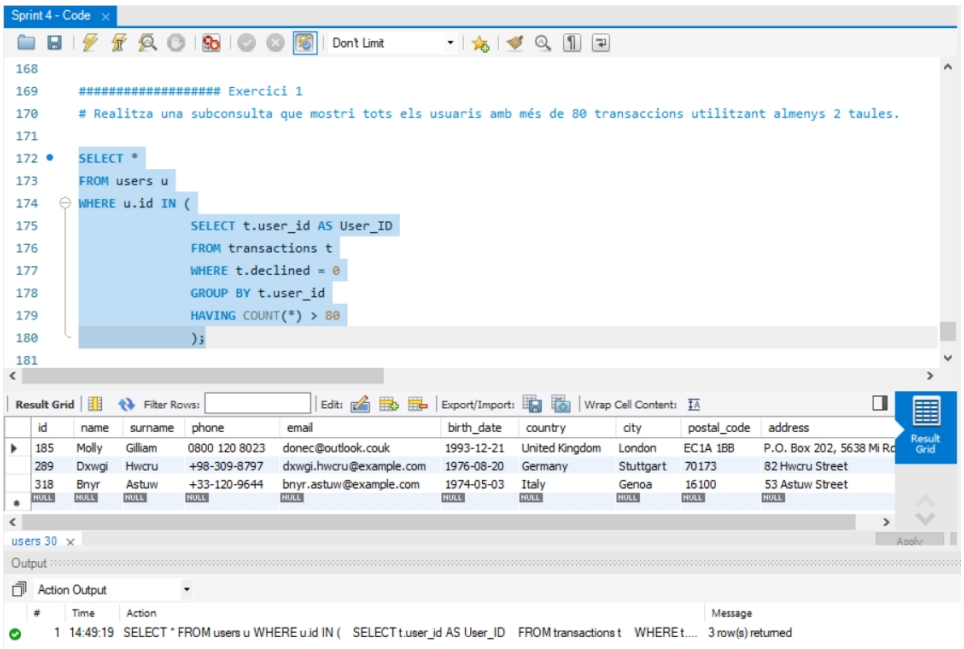

Fig 19. *Users with more than 80 transactions*

To solve this exercise, two tables, `users` and `transactions`, are used. Remember, if the command line "t.declined = 0" is omitted, there will be 4 users instead!

Exercise 2

Show the average amount per IBAN of credit cards in the company Donec Ltd. Use at least 2 tables.

Task S4.01. Database Creation

Mani Rezaeirad *p2p review partner: Damian Vallejos*

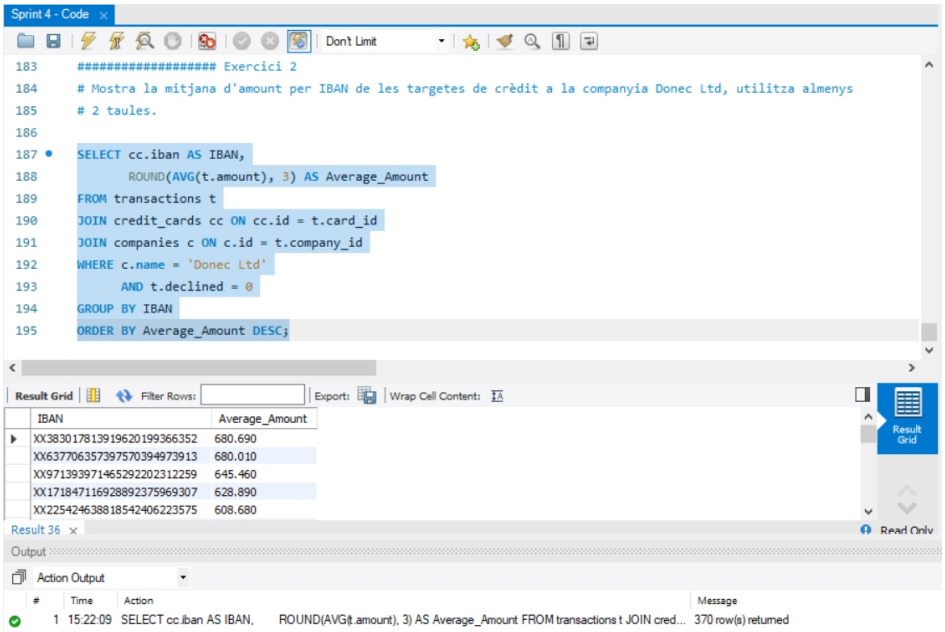

Fig 20. *Average amount per IBAN of credit cards in the company Donec Ltd*

Level 2

Create a new table that reflects the status of credit cards based on whether the last three transactions have been declined then it is inactive, if at least one is not declined then it is active. Based on this table answer:

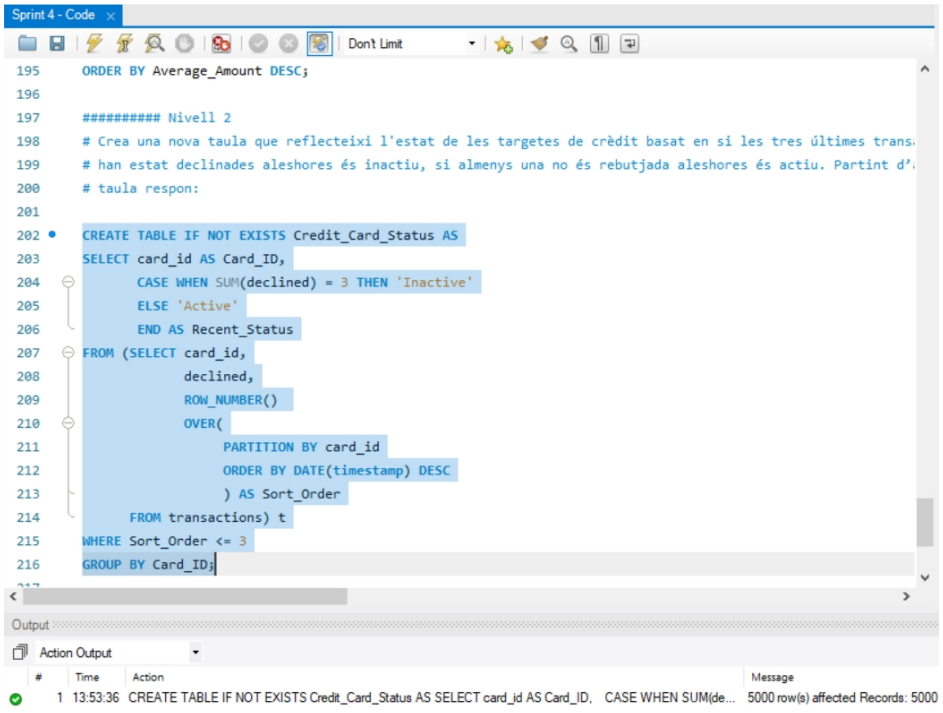

Fig 21. *Creating a new table for credit card status*

Task S4.01. Database Creation

Mani Rezaeirad *p2p review partner: Damian Vallejos*

Exercise 1
How many cards are active?

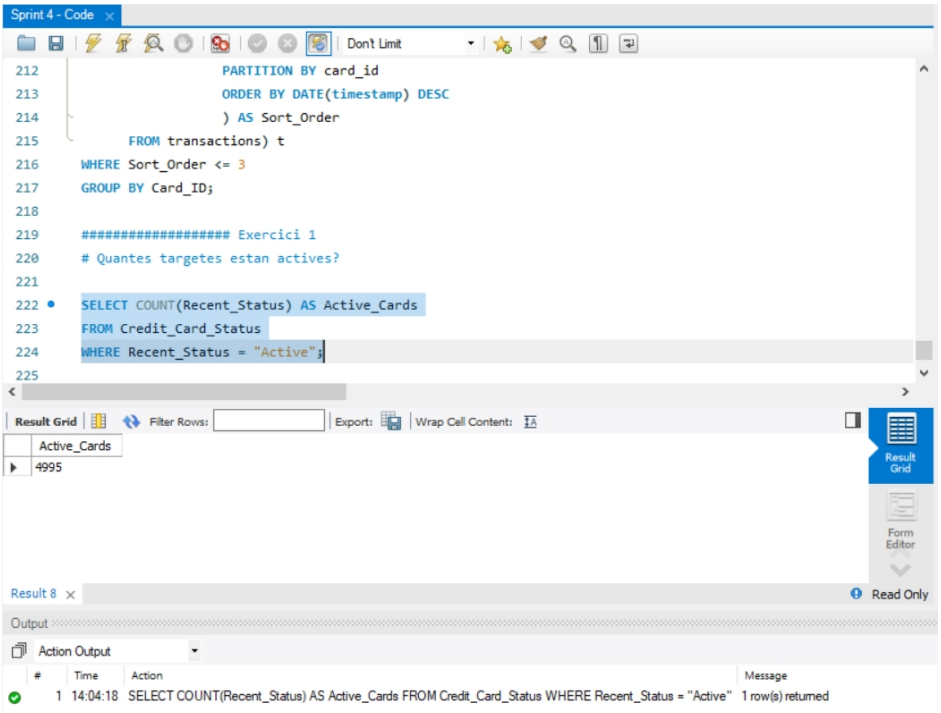

Fig 22. *Number of active credit cards*

Level 3

Create a table with which we can join the data from the new products.csv file with the created database, taking into account that from transactions you have product_ids. Generate the following query:

To do this, we need a bridge table that relates the `products` table to the `transactions` table. This phase is depicted in Figure 23. Then, we need to change the format of the `transactions.product_id` to JSON. It is done as illustrated in Figure 24.

Task S4.01. Database Creation

Mani Rezaeirad *p2p review partner: Damian Vallejos*

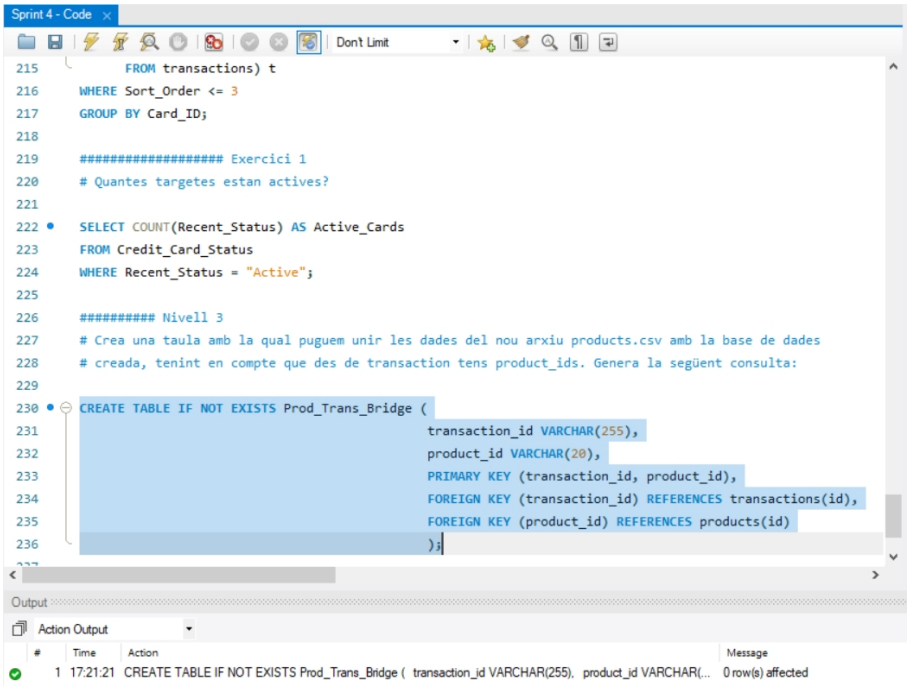

Fig 23. *Creating the bridge table between the* `users` *table and the* `transactions` *table*

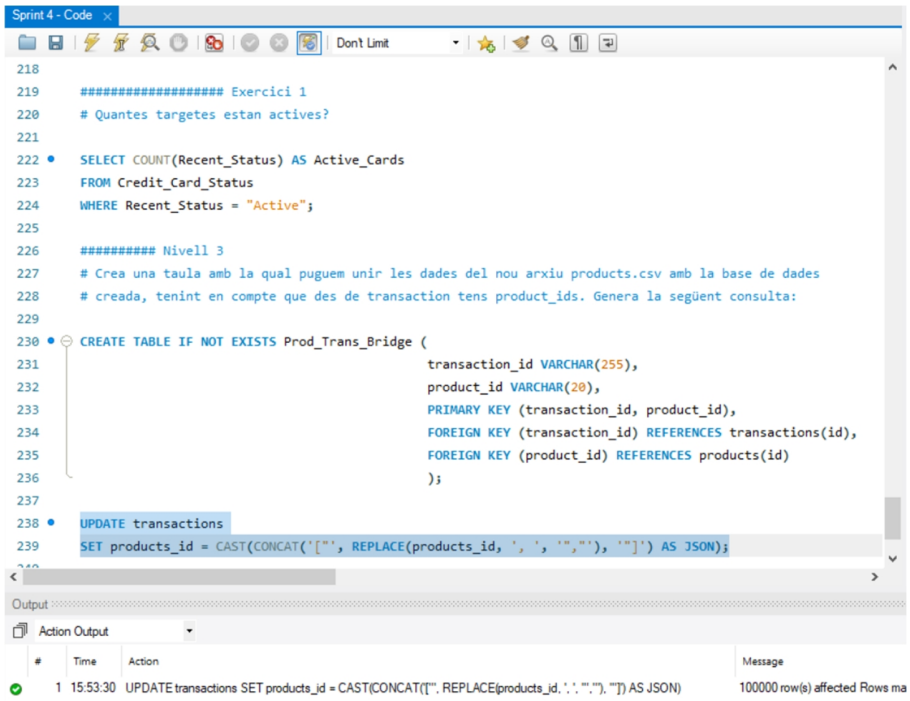

Fig 24. *Changing the format of a specific column to JSON*

Now, it is time to insert the corresponding data into our new table, using a function called JSON_TABLE, as demonstrated in Figure 25.

Task S4.01. Database Creation

Mani Rezaeirad *p2p review partner: Damian Vallejos*

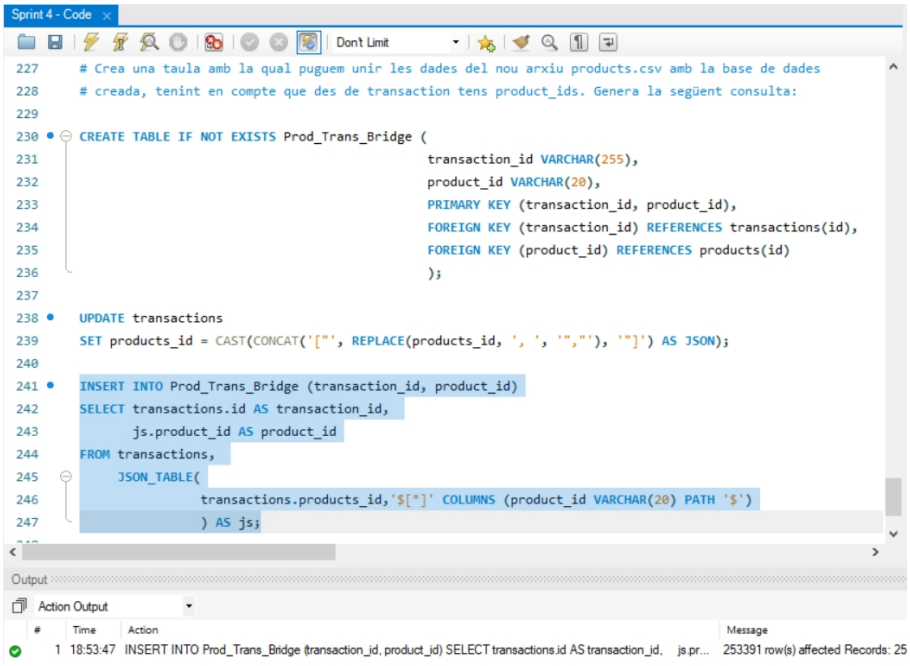

Fig 25. *Inserting data into the new table using the JSON_TABLE function*

Exercise 1

We need to know the number of times each product has been sold.

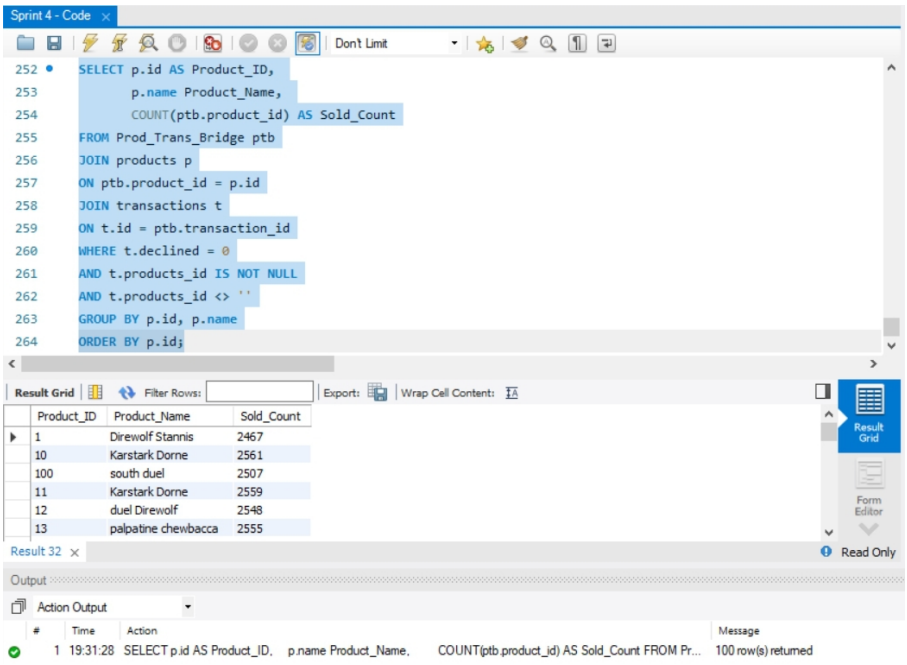

Fig 26. *The number of times each product has been sold*

Given all the available information, the number of sales per product is simply a matter of using JOIN function, as shown in Figure 26.

Task S4.01. Database Creation

Mani Rezaeirad *p2p review partner: Damian Vallejos*

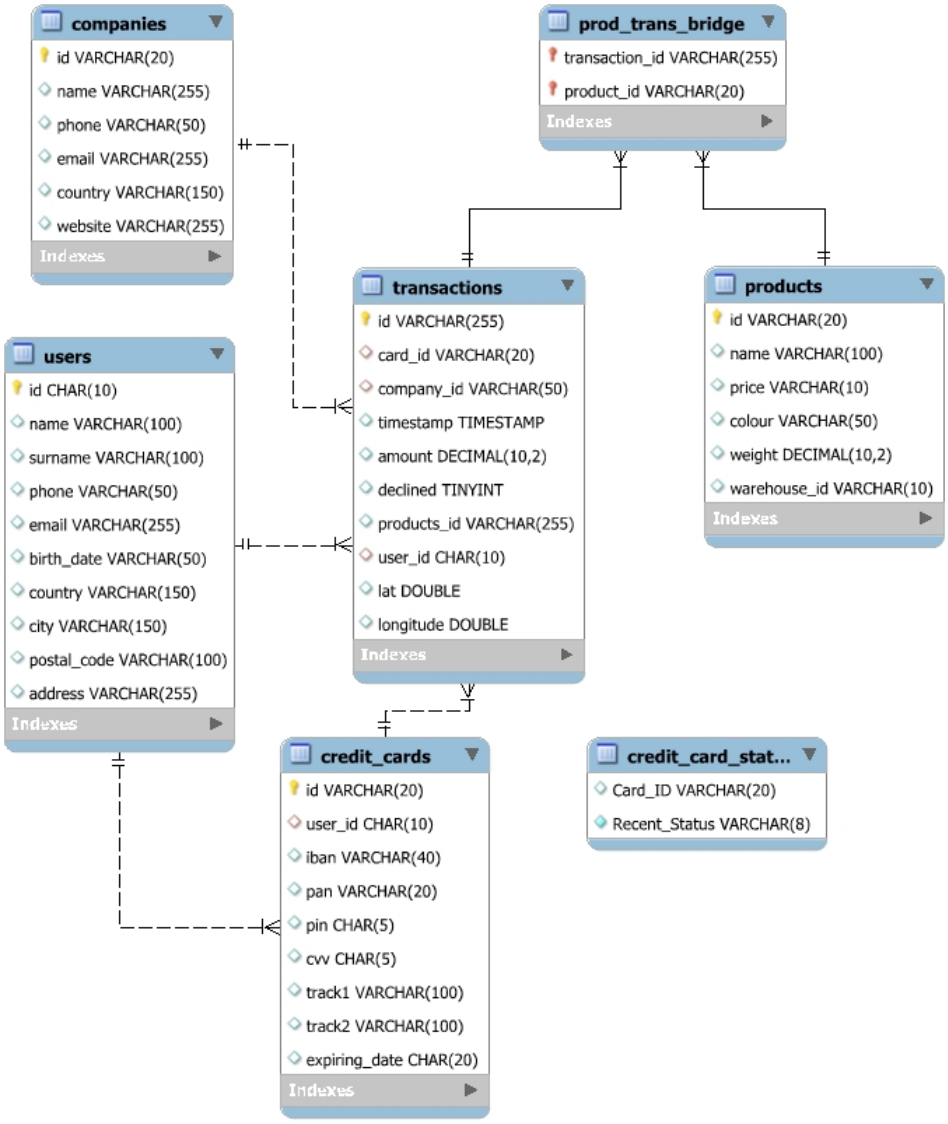

Fig 27. *Updated ER diagram of the sprint4_bbdd database*

Finally, Figure 27 shows the updated ER diagram of our database, which, unlike Figure 18, has a snowflake schema. As mentioned earlier, the `Prod_Trans_Bridge` table creates an M:N relationship between the `transactions` table and the `products` table. Note that since no exercise has asked to link the `credit_card_status` table to the `credit_cards` table, it is left without any connection.